

Here is how to containerize your application and combine it with your database using **Docker Compose** to manage the entire stack with a single command.

1. Create a `Dockerfile`

Create a file named `Dockerfile` (no extension) in your project's root folder based on your chosen application lane.

Node.js Lane

dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Use code with caution.

Python Lane

dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Use code with caution.

2. Create the `compose.yaml`

Create a `compose.yaml` file in your root folder. This maps your application inside Docker's internal networking layout to find the database at `db:5432` instead of `localhost:5432`.

(Note: Change the `ports` to `"8000:8000"` under the `api` service if you are using the Python lane).

yaml

```
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgres://postgres:dev@db:5432/tasks
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: tasks
    volumes:
      - taskdata:/var/lib/postgresql/data

volumes:
  taskdata:
```

Use code with caution.

3. Execution & Verification

Stop your manually running database container so port `5432` is free, then boot up your new multi-container architecture layout:

bash

```
# Stop old baseline instance
docker stop taskdb
```

```
# Start the unified container environment configuration
docker compose up --build
```

Use code with caution.

Checkpoint Lifecycle Testing

Verify your orchestration stack correctly preserves your table row records across container lifecycles:

bash

1. Create a couple of task entries

```
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Dockerize Stack"}' http:
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Test Persistence"}' http
```

2. Tear down the whole active runtime architecture stack

```
docker compose down
```

3. Spin the entire environment system up again

```
docker compose up -d
```

4. Check that data records persist from your named volume attachment

```
curl -i http://localhost:3000/tasks
```